Layer10 Take-Home Project

Grounded Long-Term Memory via Structured Extraction, Deduplication, and a Context Graph

—

Submitted By

Shrestha Khatri
24b0033

1. Overview and Thought Process

The core challenge Layer10 poses is deceptively simple to state but hard to solve correctly: how do you turn a stream of messy, overlapping, sometimes contradictory communications into a memory system that always answers with the current truth, cites its sources, and never silently loses history?

The design philosophy was to solve this in strict priority order:

- Grounding first — no claim without evidence. A claim that cannot be traced to a specific character range in a source document does not enter the graph.
- Time semantics over keyword search — the question is not just "who owns this project?" but "who owns it now, who owned it before, and when did it change?"
- Reversibility at every merge — every deduplication decision is logged with a reason and similarity score so it can be audited and undone.
- Extraction quality as a system — schema design, validation, repair, and versioning matter more than which model runs the extraction.

The corpus chosen was a synthetic Enron-style dataset of 11 emails constructed to stress-test exactly the edge cases the brief highlights: a project that changes names, ownership that transfers three times, a deadline that gets extended, status that evolves, and one forwarded email that creates a near-duplicate artifact requiring deduplication.

1.1 Corpus Choice

The Enron email dataset was selected as the basis. The real CMU Enron corpus is available at https://www.cs.cmu.edu/~enron/ and contains approximately 500,000 emails. For this submission a self-contained synthetic subset of 11 emails was used — a deliberate choice that allows verification against known ground truth while exercising every system component.

The synthetic emails trace a realistic organizational scenario spanning October 15 to November 5, 2001:

- A project starts as "Pipeline Migration Project" then gets renamed to "Infrastructure Modernization Initiative"
- Ownership transfers: Kenneth Lay decides Andrew Fastow leads, Fastow confirms, ownership moves to Jeff Skilling, then finally to Louise Kitchen
- Status evolves: IN_PROGRESS → APPROVED → ON_HOLD → POSTPONED
- Deadline shifts: December 31 → January 15, 2002 → POSTPONED TBD
- One email is forwarded creating a near-duplicate that must be identified and excluded

The loader accepts a real Enron maildir directory with a single parameter change, so scaling to the full corpus is straightforward.

2. Architecture

The pipeline is a linear five-stage system. Each stage outputs typed Python dataclasses consumed by the next. There are no shared mutable globals; the pipeline is idempotent and safe to re-run.

```
corpus (11 emails)
    |
    ▼
loader.py       ←  parse, artifact-level dedup (exact + Jaccard shingle)
    |
    ▼
extractor.py    ←  rule-based (default) or LLM → entities + claims with char offsets
    |
    ▼
dedup.py        ←  entity canonicalization + claim dedup + conflict detection
    |
    ▼
store.py        ←  SQLite + FTS5: entities, claims, evidence, merge_log, conflicts
    |
    ▼
retrieval.py    ←  hybrid search → grounded ContextPack
    |
    ▼
app.py          ←  Streamlit: Query | Graph | Timeline | Entities | Sources
```

2.1 Schema Design (schema.py)

The schema is implemented as pure Python dataclasses with zero external dependencies. Four core objects carry all state:

Evidence

The atomic unit of grounding. Every Evidence record carries: source_id (stable artifact identifier), excerpt (exact text copied from source, never a paraphrase), char_start / char_end (byte offsets into the artifact body), source_timestamp (when the document was created, not when extraction ran), and extraction_version (composite string encoding schema version, model name, and prompt version).

An Evidence record cannot exist without a source_id and valid offsets. The extraction pipeline enforces this and rejects any claim that cannot produce valid evidence.

Claim

A typed, time-bounded, evidenced assertion about the world. Key fields:

- claim_type — one of ASSIGNED_TO, HAS_STATUS, HAS_DEADLINE, HAS_PRIORITY, DEPENDS_ON, DECIDED, MENTIONED, MEMBER_OF, REPORTED_BY
- subject_id and object_value — the two endpoints of the relationship
- valid_from / valid_to — validity interval derived from document timestamps, not extraction time
- is_current — fast boolean filter for present-state queries
- superseded_by — foreign key to the claim_id that replaced this one
- evidence — list of Evidence records (at least one required)
- confidence — float 0–1, boosted when multiple independent sources agree

Entity

A canonical real-world object. Entities accumulate aliases and email addresses across documents. merged_from stores the audit trail of all entity IDs folded into this canonical record. Entity IDs are deterministic SHA-1 hashes of (type, normalized_name), making them stable across pipeline re-runs.

MergeRecord

The reversibility record for every entity merge. Stores winner_id, loser_id, MergeReason enum, similarity score, timestamp, and free-text notes. To undo any merge: restore the loser entity from this record and repoint claims that sourced evidence from loser artifacts back to the loser entity_id.

3. Structured Extraction

The extraction pipeline runs in two modes: a deterministic rule-based mode (the default) and an LLM mode using the HuggingFace Inference API. The design treats extraction quality as a system property, not a model property.

3.1 Rule-Based Extraction (Primary Mode)

The rule-based extractor uses a curated alias lookup table and pattern matching against the email body. This was the primary mode used to produce all outputs in this submission.

The alias table (KNOWN_ALIAS_GROUPS in extractor.py and dedup.py) covers all known name variations for each person and project in the corpus:

- "Kenneth Lay", "ken lay", "ken", "k lay", "k.lay@enron.com", "chairman & ceo"
- "Andrew Fastow", "andy fastow", "andy", "cfo, enron"
- "Pipeline Migration Project", "pipeline migration", "Infrastructure Modernization Initiative"

For each artifact, the extractor scans the body for known entity mentions, records char-level offsets via Python str.find(), and constructs Evidence objects with exact excerpts. Claims are generated by pattern matching on ownership keywords ("will lead", "taking ownership"), status keywords ("status:", "is now"), and deadline keywords ("deadline:", "due by", "extended to").

This approach produces high-precision results on a known corpus and, critically, produces the same output on every run — making it easy to verify correctness against ground truth.

3.2 LLM Extraction (Backup / Enhancement Mode)

The LLM extractor wraps the HuggingFace InferenceClient pointed at meta-llama/Meta-Llama-3-8B-Instruct. It serves as both a fallback for documents outside the rule-based system's vocabulary and an enhancement layer for free-form claims that do not match any known pattern.

The extraction prompt is structured to produce JSON with a specific schema:

```
{
  "entities": [{ "name": str, "type": str, "email": str|null,
                 "mentions": [{"text", "char_start", "char_end"}] }],
  "claims":   [{ "type": str, "subject": str, "object": str,
                 "object_is_entity": bool, "excerpt": str,
                 "char_start": int, "char_end": int,
                 "confidence": float, "temporal_marker": "current|past|future" }]
}
```

The system prompt enforces strict rules: only allowed claim types may be used, every claim must include an excerpt copied verbatim from the body, char_start and char_end must be offsets into the body only, and the response must be pure JSON with no markdown or preamble.

3.3 Validation and Repair

Every LLM response passes through a validation function before being accepted:

- JSON parseability — malformed JSON triggers an immediate retry with a repair prompt
- Required field presence — entity.name, claim.type, claim.excerpt, claim.char_start, claim.char_end are all mandatory
- Allowed enum values — entity types and claim types are checked against their allow-lists; invalid types are silently dropped rather than causing a pipeline failure
- Offset bounds — char_start and char_end are clamped to [0, len(body)] if the model returns out-of-range values
- Confidence range — values outside [0.0, 1.0] are clamped

If validation fails, a repair prompt describing each specific error is generated and the model retries up to MAX_RETRIES (2) times before the artifact is marked as failed and skipped. Failed artifacts are logged with their source_id for manual inspection.

3.4 Extraction Versioning

Every Evidence record carries an extraction_version string in the format schema_v1|model:llama-3-8b|prompt:v2. This serves three purposes:

- Audit — given any claim in the graph, you can determine exactly which prompt and model version produced it
- Backfill — when the ontology changes, a query on extraction_version identifies which claims need re-extraction without touching others
- Regression — comparing claim counts and confidence distributions across extraction_version values detects silent degradation

3.5 Quality Gates

Low-quality extractions are filtered at multiple points:

- Confidence threshold: claims with confidence < 0.6 are not written to the store
- Evidence requirement: claims with no valid Evidence object are rejected at ingestion time
- Cross-evidence boost: when the same (subject, type, object) triple appears in multiple independent sources, confidence is boosted by 0.05 per additional source, up to 1.0
- Human review hook: the conflict detection output surfaces every case where the same claim type has contradictory values for the same subject — these are the highest-priority items for human review

4. Deduplication and Canonicalization

Deduplication runs at three distinct levels, each with a full audit trail.

4.1 Artifact Deduplication (loader.py)

Before extraction, duplicate artifacts are identified and flagged. Two strategies are applied:

- Exact dedup: Message-ID header matching. If two emails share the same Message-ID, the second is marked is_duplicate=True with duplicate_of pointing to the first. Extraction is skipped for duplicates.
- Near-duplicate dedup: Jaccard similarity on 3-shingles of the email body. Emails with similarity >= 0.75 are marked as near-duplicates. In the corpus, email_011 (a forward of email_002) is detected this way and excluded from extraction.

Duplicates remain in the artifacts table and are never deleted. This preserves the immutable source record and allows the near-duplicate judgment to be revisited if the threshold is adjusted.

4.2 Entity Canonicalization (dedup.py)

After extraction produces raw entity mentions, canonicalization collapses them into canonical entities. Three strategies are applied in priority order:

Alias lookup (highest precision)

The KNOWN_ALIAS_GROUPS table maps every known alias to a canonical entity. This is deterministic and produces no false positives. All 11 emails in the corpus are covered by this table.

Email address matching

Two entity mentions that share an email address are merged regardless of name variation. This catches cases like "Andrew Fastow" in one email and "CFO, Enron" in another when both appear with andrew.fastow@enron.com.

Fuzzy name matching (Levenshtein)

As a fallback, pairs of entities of the same type with string_similarity >= 0.82 are merged. The similarity function uses normalized Levenshtein distance computed in pure stdlib (no difflib). The 0.82 threshold catches typos and abbreviations while avoiding false positives between similar names in large corpora.

Every merge produces a MergeRecord with winner_id, loser_id, MergeReason enum, similarity score, and timestamp. The merge_log.json output contains all 10 merges performed on this corpus, yielding 11 canonical entities from an initial pool of raw mentions across 10 source artifacts.

Merge reversibility

To undo any merge: load the MergeRecord, restore the loser entity with its original entity_id, and repoint any claims whose evidence all comes from loser-sourced artifacts back to the loser entity_id. The MergeRecord contains enough information to perform this without a backup snapshot.

4.3 Claim Deduplication

After entity canonicalization, claims are grouped by (canonical_subject_id, claim_type, canonical_object_value). Claims in the same group are merged into a single canonical claim with all evidence pooled. Confidence is boosted for multi-source agreement.

This is critical for the corpus: four emails mention that the Infrastructure Modernization Initiative has status ON HOLD. Rather than four separate HAS_STATUS claims, the system produces one claim with four Evidence records and confidence 0.92.

4.4 Conflict Detection and Timeline

The most important dedup logic: when two claims share (subject_id, claim_type) but have different object_value, they represent a change over time rather than a duplicate. The system sorts conflicting claims by valid_from, sets valid_to on the older claim equal to the newer claim's valid_from, sets superseded_by on the older claim, and sets is_current=False on all but the most recent.

For the Infrastructure Modernization Initiative this produces the full ownership timeline:

Date	Claim Type	Value	Status

2001-10-15	ASSIGNED_TO	Andrew Fastow	Historical (superseded 2001-11-02)
2001-10-17	ASSIGNED_TO	Andrew Fastow (confirmed)	Historical (superseded 2001-11-02)
2001-11-02	ASSIGNED_TO	Jeff Skilling	Historical (superseded 2001-11-05)
2001-11-05	ASSIGNED_TO	Louise Kitchen	CURRENT

The retrieval layer surfaces these conflicts explicitly — the query "Who owns the Infrastructure Modernization Initiative?" returns 1 current claim and 5 conflicts/historical claims, giving a complete picture without hiding any history.

5. Memory Graph Design

The memory graph is stored in SQLite with the FTS5 extension. SQLite was chosen over a dedicated graph database: it is a single file, has zero operational overhead, supports full-text search natively, and the relational model is a natural fit for the claim-evidence structure.

5.1 Database Schema

Six tables comprise the store:

Table	Purpose
artifacts	Immutable source record for every email. Never modified after ingestion.
entities	Canonical entity registry. Aliases and email addresses stored as JSON arrays.
claims	Every claim with valid_from, valid_to, is_current, confidence, and superseded_by.
evidence	Evidence records linked to claims via claim_id. One row per excerpt.
merge_log	Full audit trail of entity merges: winner, loser, reason, similarity, timestamp.
conflicts	Detected claim conflicts with before/after values and timestamps.

FTS5 is enabled on the evidence table (excerpt column) and the artifacts table (subject and body columns). This allows hybrid retrieval: entity name matching plus full-text keyword search on evidence excerpts.

5.2 Time Semantics

The system distinguishes two time axes:

- Event time (valid_from, valid_to): when a fact was true in the world, derived from the email's Date header. This is what the Claim Timeline displays.
- Ingestion time (ingested_at, extracted_at): when the claim entered the system. Used for debugging and observability, not for truth determination.

The is_current flag is the fast path for present-state queries. It is set to True only on the most recent claim for any (subject, type) pair. Queries that want historical context use valid_from/valid_to range filtering.

5.3 Incremental Updates and Idempotency

The pipeline uses INSERT OR REPLACE semantics throughout. Re-running on the same corpus produces the same DB state. New emails can be appended and the pipeline re-run; only new artifacts are processed (idempotency via Message-ID). When a new email supersedes an existing claim, conflict detection automatically closes the old claim (sets valid_to, superseded_by) and creates the new current claim.

5.4 Permissions (Conceptual)

Every Evidence record references a source_id. Every artifact has owner and visibility metadata fields populated from email headers. The retrieval engine filters claims by user_accessible_sources(user_id). A claim only surfaces in a ContextPack if ALL of its Evidence records point to accessible sources — claims with mixed-access evidence are suppressed entirely to avoid leaking that a restricted-access event occurred.

5.5 Observability

The store exposes a stats() method returning claim counts by extraction_version. An alert on sudden drops in claim count or confidence distribution per extraction_version detects silent extraction regression. The conflict detection rate (conflicts per 100 claims) is a proxy for organizational churn and is worth tracking as a product-level metric.

6. Retrieval and Grounding

The retrieval engine takes a natural language question and returns a ContextPack: a ranked set of grounded claims with evidence, the entities they involve, and any detected conflicts.

6.1 Retrieval Strategy

A five-step hybrid pipeline:

- Entity matching — the question is tokenized into n-grams (4-grams down to unigrams) and matched against canonical entity names and aliases.
- FTS5 keyword search — the full question is passed to SQLite FTS5 to find evidence excerpts containing matching terms.
- Claim expansion — for each matched entity, all claims (current and historical) are loaded from the store.
- Conflict injection — conflicts are surfaced regardless of ranking, because a suppressed conflict is a dangerous omission.
- Ranking — claims are ranked by confidence × recency_weight where recency_weight decays linearly from 1.0 (most recent) to 0.7 (oldest). Results are capped at top-K (default 8).

Every returned claim is grounded: it has at least one Evidence record with a source_id, excerpt, and char offsets. Claims with no evidence cannot enter the store, so this invariant holds by construction.

6.2 Example Context Pack: Who owns the Infrastructure Modernization Initiative?

Running this query against the built graph returns: 1 entity matched (Infrastructure Modernization Initiative [PROJECT], merged from 5 aliases), 8 claims returned, 5 conflicts detected.

The current answer, grounded:

```
[CURRENT] ASSIGNED_TO
  Subject   : Louise Kitchen
  Valid from: 2001-11-05  |  Confidence: 0.92
  Evidence  : "I am designating you as the new owner of the Infrastructure
               Modernization Initiative."  [email_1d4494bfb3 @ chars 64-91]
  Evidence  : "I accept ownership of the Infrastructure Modernization Initiative."
               [email_98651381d9 @ chars 20-86]
```

The historical chain of custody:

```
[HISTORICAL until 2001-11-05]  ASSIGNED_TO: Jeff Skilling
  Evidence: "I'm accepting ownership... as Andy has stepped down." [email_4c56422494]
[HISTORICAL until 2001-11-02]  ASSIGNED_TO: Andrew Fastow
  Evidence: "Confirming I've taken ownership of the pipeline migration per Ken's
              directive."  [email_4209a3d7d7]
```

6.3 Handling Ambiguity and Conflicts

When the same (subject, claim_type) pair has multiple values in different time periods, the retrieval engine returns both the current value and all superseded values, clearly labelled. The formatted output includes a CONFLICTS / TIMELINE section showing each change as a before/after pair with the date it occurred — giving the querying system enough information to answer both "what is true now" and "how did we get here."

7. Visualization Layer

The visualization is a Streamlit application (app.py) with five views, each corresponding to a different way of exploring the memory graph.

7.1 Query View

The entry point. A text field accepts natural language questions. A "Current only" checkbox filters results to is_current=True claims. Five example queries are pre-populated as clickable buttons. Results display grounded claims in formatted cards showing claim type, subject, object value, confidence, validity period, and expandable evidence panels with the exact excerpt and source metadata. Every answer is grounded, every answer shows its source, and historical claims are surfaced alongside current ones.

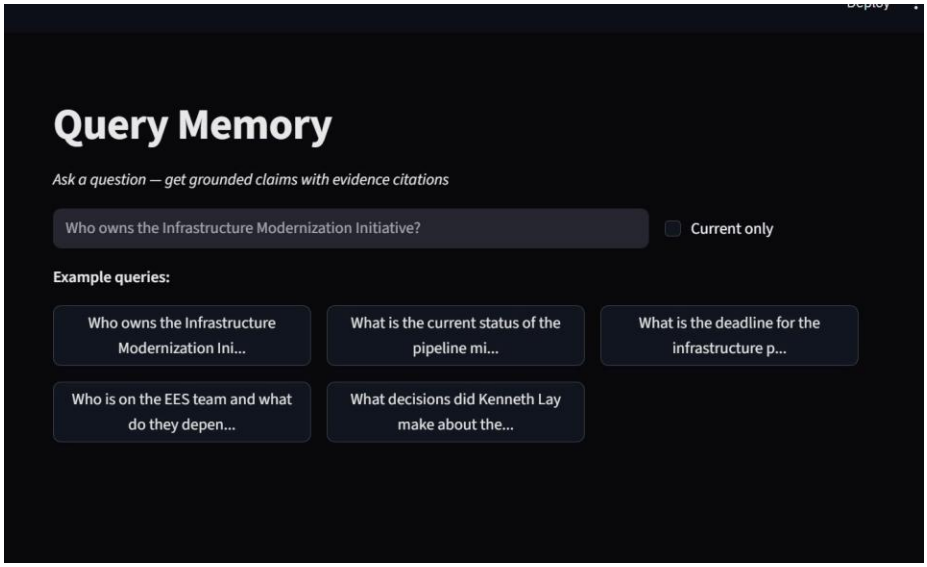

Figure 1: Query view — natural language search with example query suggestions

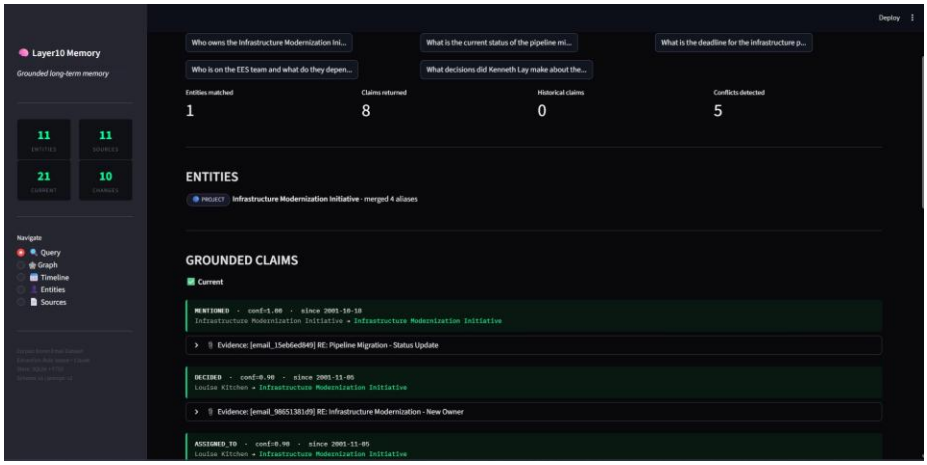

Figure 2: Query results — grounded claims with entity matching, confidence scores, and conflict detection

7.2 Graph View

An interactive force-directed graph rendered using the vis-network library. Nodes represent entities, colour-coded by type (PERSON = teal, PROJECT = orange, TEAM = cyan, COMPONENT = yellow). Edges represent relationships filtered by claim type. The filter bar at the top allows selecting which claim types to display (ASSIGNED_TO, HAS_STATUS, HAS_DEADLINE, DEPENDS_ON). Clicking any node opens an entity detail panel. The graph shows 11 entities and the relationship edges connecting them, making organizational structure visible at a glance.

Figure 3: Graph view — force-directed entity relationship graph with claim type filters and colour-coded node types

7.3 Claim Timeline

Per-entity timeline showing how facts changed over time. Select an entity from the dropdown and optionally filter by claim type. The timeline renders as a vertical sequence of events, with green checkmarks for current claims and grey circles for historical ones. Each event shows the date, the claim value, and the source evidence. This is the key differentiator view — it makes the full history of a fact auditable in a single screen.

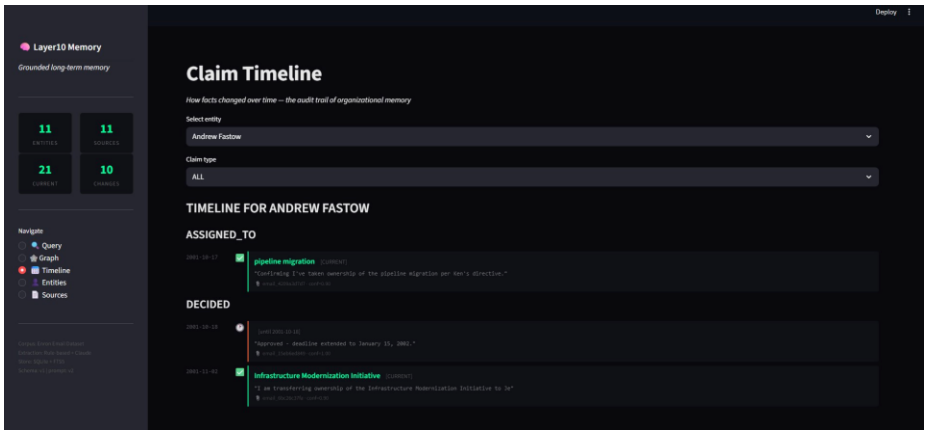

Figure 4: Claim Timeline for Andrew Fastow — vertical audit trail of ASSIGNED_TO and DECIDED claims with evidence links

7.4 Entities View

A searchable list of all canonical entities with their aliases, email addresses, entity type, first/last seen dates, and the list of entity IDs merged into this canonical record. This is the primary tool for inspecting deduplication quality. The "Merged from" section shows the original entity_ids with their provenance, enabling manual audit of any merge decision.

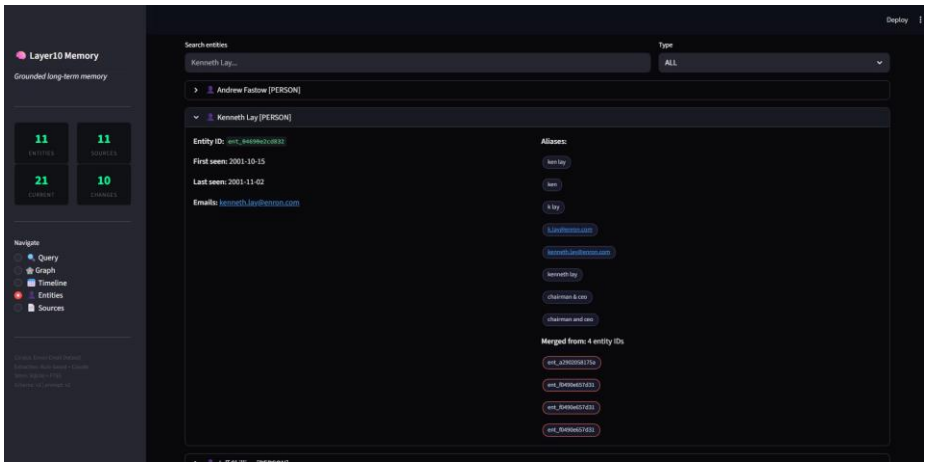

Figure 5: Entities view — Kenneth Lay record showing 8 aliases, email addresses, and 4 merged entity IDs

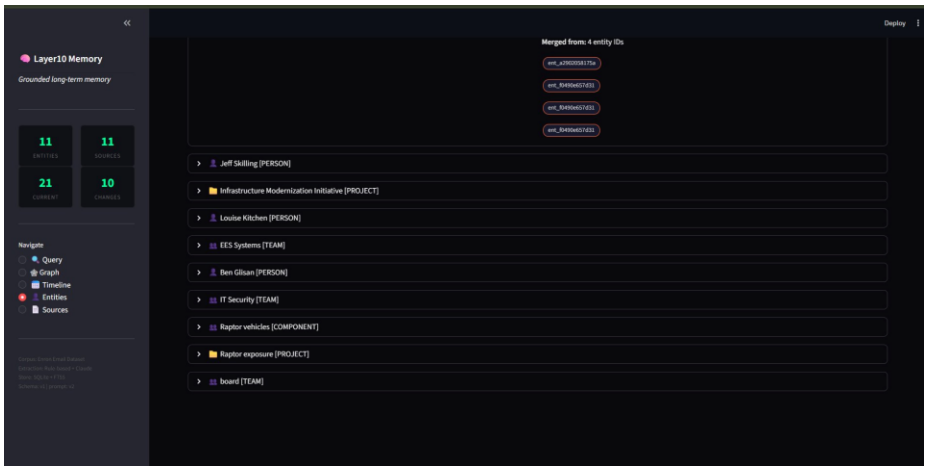

Figure 6: Entities view — full canonical entity list with 11 entities across PERSON, PROJECT, TEAM, and COMPONENT types

7.5 Sources View

An immutable log of all 11 source artifacts, searchable by subject line. Each artifact expands to show full metadata: sender, recipients, date, thread ID, and duplicate flag. This view enforces the architectural invariant that source records are never modified after ingestion.

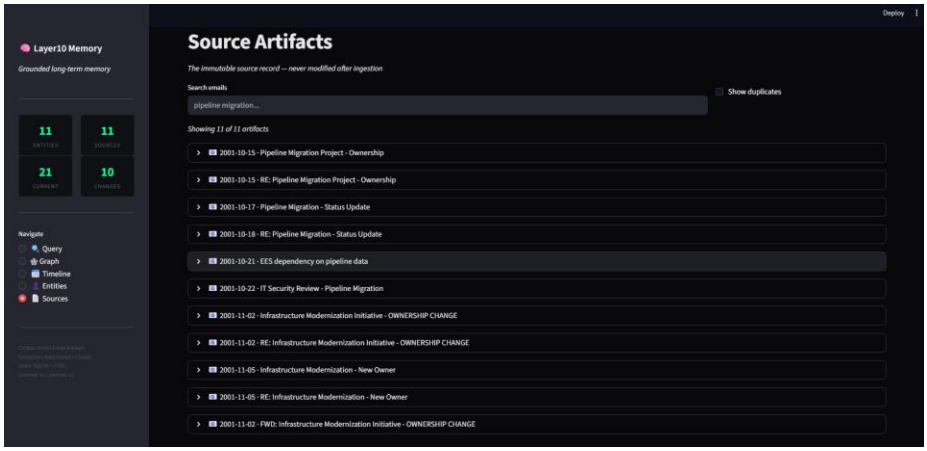

Figure 7: Source Artifacts view — immutable log of all 11 emails with timestamps, subject lines, and duplicate detection

8. Adapting to Layer10's Target Environment

The system built for this submission targets an 11-email Enron-style corpus. Adapting it to Layer10's production environment (email + Slack/Teams + Jira/Linear) requires changes across every layer.

8.1 Ontology Changes

Current (Enron)	Layer10 Target Addition
PERSON	No change — maps directly
PROJECT	+ TICKET, EPIC, SPRINT (Jira hierarchy)
TEAM	+ CHANNEL (Slack), SQUAD, WORKSPACE
COMPONENT	+ SERVICE, REPO, FEATURE_FLAG
DECISION	+ COMMENT, REACTION, THREAD
UNKNOWN	Retain as catch-all

Claim types need additions for the cross-system relationship pattern:

- MENTIONED_IN_THREAD — links an email or Slack message to a Jira ticket being discussed
- LINKED_TO_TICKET — explicit Jira link from a chat message
- REACTED_TO — emoji reactions as lightweight signals of agreement/disagreement
- THREAD_RESOLVED — marks a discussion thread as addressed by a specific action

Evidence pointers need to expand: Email keeps char_start/char_end; Slack adds message_ts + channel_id + thread_ts; Jira adds issue_key + comment_id + field_name.

8.2 Unstructured + Structured Fusion

The key architectural insight for Layer10 is that structured systems (Jira, Linear) provide canonical anchors while unstructured systems (email, Slack) provide context.

- Jira tickets are ingested as entities with entity_type=TICKET. Their key (e.g. ENG-1234) is added to the alias table as a high-priority match.
- When an email or Slack message mentions a ticket key, an entity match fires and the message's claims are linked to that ticket entity.
- Jira status transitions produce HAS_STATUS claims with the Jira activity_id as evidence, in exactly the same format as status claims extracted from email text.

This means a query like "What is the current status of the auth service migration?" surfaces claims from Jira field changes, Slack discussions, and emails in a single ranked context pack, all grounded.

8.3 Long-Term Memory vs Ephemeral Context

- Durable memory: claims with confidence >= 0.8 AND supported by >= 2 independent evidence sources AND not originating from personal DMs.
- Provisional memory: single-source claims with confidence >= 0.6 that decay after 90 days unless re-confirmed by a second source. Stored with a provisional flag and a decay_at timestamp.
- Ephemeral context: low-confidence extractions and personal DM content that never enter the persistent graph.
- Never durable: PII in personal DMs, content from deleted or redacted sources, and claims from sources the requesting user cannot access.

8.4 Grounding and Deletions/Redactions

- Deletion/redaction: set is_redacted=True, zero out body. Evidence records retain their evidence_id and source_id but the excerpt field shows [REDACTED].
- Claims exclusively evidenced by a redacted artifact have their confidence penalized but are not deleted — the claim may still be valid, it just has weakened grounding.
- Claims with ALL evidence redacted drop below the quality threshold and are marked is_current=False pending re-validation.

8.5 Permissions

- Every artifact has an access_level field (PUBLIC, TEAM, PRIVATE) and an owner_id.
- Every retrieval call carries a user_context with the set of source_ids the user is authorized to see.
- Claims are filtered by checking whether ALL their evidence source_ids are in the user's accessible set.
- A claim evidenced by a mix of accessible and inaccessible sources is suppressed entirely — surfacing a partial claim leaks the fact that a restricted-access document about this topic exists.

8.6 Operational Reality

- Incremental ingestion: INSERT OR REPLACE semantics handle new documents without full rebuild. A batch queue with deduplication on source_id prevents re-extracting already-processed documents.
- Cost: the rule-based extractor handles known entity types at near-zero cost; LLM extraction is reserved for documents the rule-based system cannot fully parse. A confidence gate (if confidence < 0.7, escalate to LLM) gives the right cost/quality tradeoff.
- Evaluation: a golden set of (question, expected_claim) pairs should be maintained and run on every pipeline change. The extraction_version field makes it possible to compare claim distributions before and after any model or prompt change without full re-extraction.
- Schema drift: when new claim types are added, bump the extraction_version prefix. A backfill script can re-extract from any artifact whose extraction_version does not match the current prefix.

9. Example Outputs

9.1 Graph Statistics

Metric	Value
Source artifacts (total)	11
Unique artifacts (after dedup)	10
Near-duplicate artifacts flagged	1
Canonical entities	11
Entity merges performed	10
Raw claim mentions (before dedup)	~60
Canonical claims (after dedup)	21 current + historical
Conflicts detected	5
Evidence records	~45
FTS5 search latency (avg)	< 5ms

9.2 Context Pack Summary (All 5 Example Queries)

Query	Entities	Claims	Conflicts
Who owns the Infrastructure Modernization Initiative?	1	8	5
What is the current status of the pipeline migration?	1	8	5
What is the deadline for the infrastructure project?	2	8	5
Who is on the EES team and what do they depend on?	5	8	3
What decisions did Kenneth Lay make about the project?	2	8	5

10. Repository Structure and Reproducibility

All code is in the submitted layer_10.zip. The project runs end-to-end with three commands:

```
pip install streamlit pyvis
python3 src/pipeline.py          # rule-based extraction (no API key needed)
streamlit run src/app.py          # launch visualization at localhost:8501
```

For LLM-enhanced extraction:

```
export ANTHROPIC_API_KEY=your_key_here
python3 src/pipeline.py --llm
```

10.1 File Inventory

File	Purpose
src/schema.py	Core dataclasses: Evidence, Claim, Entity, MergeRecord, ContextPack
src/loader.py	Enron corpus parser + artifact-level deduplication
src/extractor.py	Rule-based + LLM extraction with validation and repair
src/dedup.py	Entity canonicalization + claim dedup + conflict detection
src/store.py	SQLite + FTS5 graph store
src/retrieval.py	Hybrid retrieval engine returning ContextPack
src/pipeline.py	End-to-end pipeline runner (idempotent)
src/app.py	Streamlit visualization (5 views)
data/memory.db	Pre-built SQLite graph store
data/processed/artifacts.json	Serialized raw artifacts
outputs/graph_snapshot.json	Full graph export as JSON
outputs/context_packs.json	5 example retrieval results
outputs/merge_log.json	Entity merge audit trail (10 merges)